

UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS
(Universidad del Perú, DECANA DE AMÉRICA)
FACULTAD DE INGENIERÍA ELECTRÓNICA Y ELÉCTRICA



DOCENTE:

Villafuerte Barreto, Hernan Oswaldo

CURSO:

Circuitos Digitales

INTEGRANTES:

Melendez Romero Jose Francesco

Sabrera Ortiz, Angie Roxana

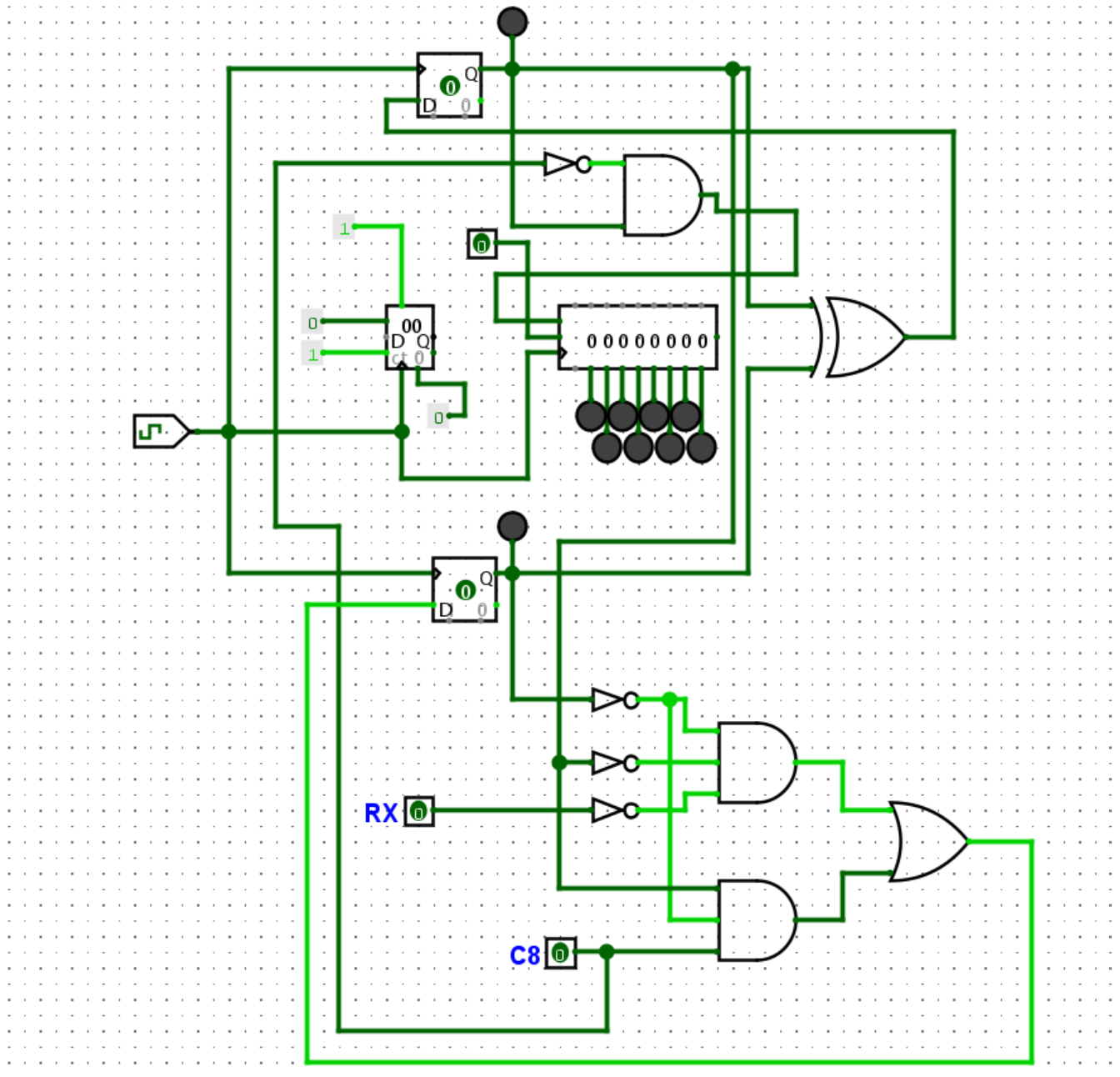
Berrocal Casanca Esaú Andree

LIMA - PERÚ

2026

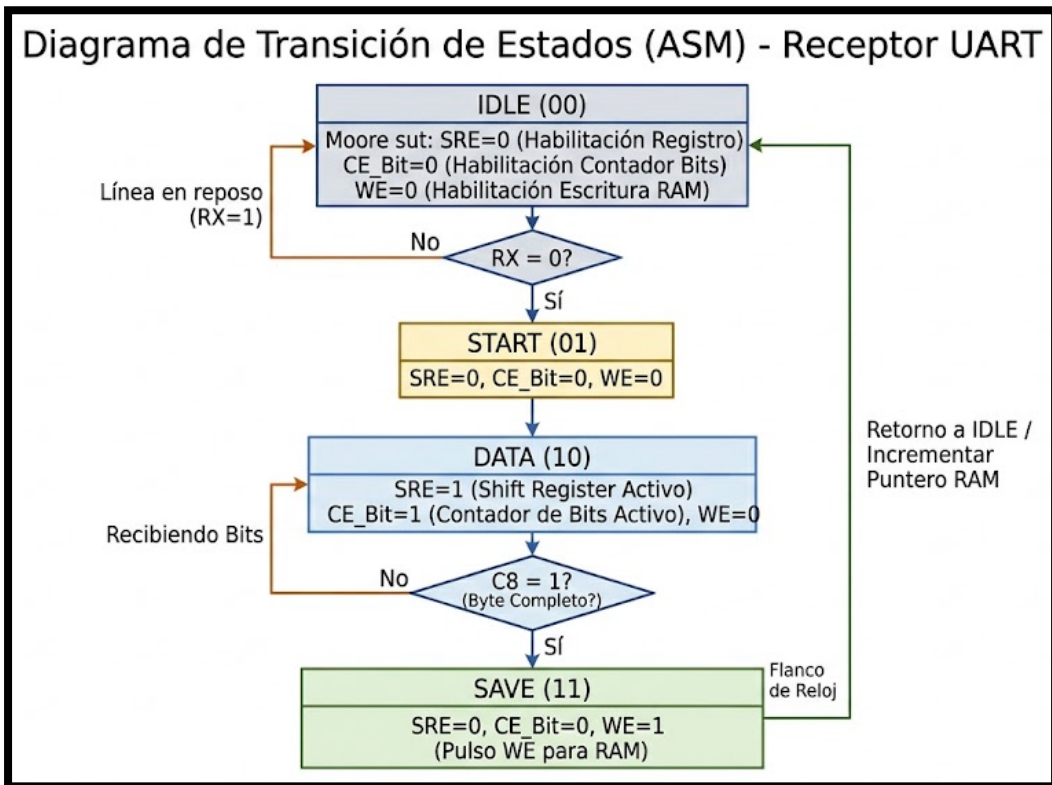
Implementación de un sistema digital de validación de integridad por paridad longitudinal con banco de registros direccionables

1. Diagrama lógico/esquemático completo



El esquema lógico implementado consta de una Unidad de Control (FSM de 2 biestables) y una Ruta de Datos. La entrada serie RX alimenta tanto a la FSM para la detección del bit de inicio, como al Registro de Desplazamiento de 8 bits para la captura de la trama. Un contador de 4 bits actúa como puntero de direcciones, conectado directamente a un Decodificador de 4 a 16 líneas. Este decodificador habilita físicamente las posiciones de la memoria RAM (16 x 8 bits) para almacenar el byte paralelo una vez que la FSM activa la señal de control *Write Enable*.

2. Diagrama de transición de estados (ASM), tablas de verdad/transición



La Unidad de Control se rige por la siguiente tabla de transición para los biestables Q1 (MSB) y Q0 (LSB):

Estado Actual (Q1Q0)	Nombre	Entrada RX	Entrada C8	Estado Siguiente (D1D0)
00	IDLE	1	X	00
00	IDLE	0	X	01
01	START	X	X	10
10	DATA	X	0	10
10	DATA	X	1	11
11	SAVE	X	X	00

Ecuaciones Lógicas de Estado Siguiente:

$$D_1 = Q_1 \oplus Q_0$$

$$D_0 = (\overline{Q_1} \cdot \overline{Q_0} \cdot \overline{RX}) + (Q_1 \cdot \overline{Q_0} \cdot C_8)$$

3. Mapa de memoria o mapa de registros

El sistema utiliza un bloque de memoria RAM con 16 posiciones de 8 bits cada una. El decodificador combinacional traduce las 4 líneas del bus de direcciones (A_3, A_2, A_1, A_0) provenientes del contador en una única señal de habilitación de palabra (WL_n).

Bus de Direcciones (A3A2A1A0)	Dirección Hexadecimal	Salida del Decodificador Activa	Función
0	0x0	WL0	Almacena Trama 1
1	0x1	WL1	Almacena Trama 2
10	0x2	WL2	Almacena Trama 3
11	0x3	WL3	Almacena Trama 4
100	0x4	WL4	Almacena Trama 5
101	0x5	WL5	Almacena Trama 6
110	0x6	WL6	Almacena Trama 7
111	0x7	WL7	Almacena Trama 8
1000	0x8	WL8	Almacena Trama 9
1001	0x9	WL9	Almacena Trama 10
1010	0xA	WL10	Almacena Trama 11
1011	0xB	WL11	Almacena Trama 12
1100	0xC	WL12	Almacena Trama 13
1101	0xD	WL13	Almacena Trama 14
1110	0xE	WL14	Almacena Trama 15
1111	0xF	WL15	Almacena Trama 16 (Tope del buffer circular)

Al llegar a la dirección 0xF (Trama 16), el siguiente incremento del contador de 4 bits provocará un desbordamiento natural (*overflow*) regresando automáticamente el bus de direcciones a 0000. Esto garantiza la sobrescritura de los datos más antiguos, cumpliendo con el comportamiento diseñado de un buffer circular (FIFO) sin requerir lógica de reseteo adicional.

4. Función del decodificador:

Asegura que los datos presentes en el bus paralelo del registro de desplazamiento se escriban exclusivamente en la fila de la memoria correspondiente al valor actual del contador, activándose únicamente bajo la señal asíncrona Write Enable proveniente del estado 11 de la FSM.

5. Código HDL o pseudocódigo estructurado

El siguiente código estructurado en C++ emula la arquitectura lógica del hardware diseñado, preparado para su validación en el ESP32:

```
1 // Definición de pines y variables de estado
2 const int rxPin = 4; // Entrada serial RX
3 const int c8Pin = 5; // Señal de fin de conteo C8
4 uint8_t q1 = 0, q0 = 0; // Biestables de la FSM (Estado inicial 00: IDLE)
5
6 // Emulación de Memoria y Ruta de Datos
7 uint8_t shiftRegister = 0; // Registro de desplazamiento de 8 bits
8 uint8_t ramBuffer[16]; // Memoria RAM de 16x8 bits
9 uint8_t addressCounter = 0; // Contador de 4 bits (Puntero de direcciones)
10
11 void setup() {
12     pinMode(rxPin, INPUT_PULLUP);
13     pinMode(c8Pin, INPUT_PULLUP);
14     Serial.begin(115200);
15 }
16
17 void loop() {
18     bool rx = digitalRead(rxPin);
19     bool c8 = digitalRead(c8Pin);
20
21     // Decodificador de la FSM (Switch-Case basado en las ecuaciones lógicas)
22     uint8_t currentState = (q1 << 1) | q0;
23
24     switch (currentState) {
25         case 0: // IDLE (00)
26             if (rx == 0) { // Detección del bit de Start
27                 q1 = 0; q0 = 1; // Salto a START (01)
28             }
29             break;
30
31         case 1: // START (01)
32             q1 = 1; q0 = 0; // Transición incondicional a DATA (10)
33             break;
34
35         case 2: // DATA (10) - Habilitación del Registro de Desplazamiento
36             // Captura de datos emulada (Desplazamiento a la izquierda)
37             shiftRegister = (shiftRegister << 1) | rx;
38
39             if (c8 == 1) { // Si se completan los 8 bits
40                 q1 = 1; q0 = 1; // Salto a SAVE (11)
41             }
42             break;
43
44         case 3: // SAVE (11) - Write Enable activado
45             // El decodificador selecciona la posición addressCounter para guardar el byte
46             ramBuffer[addressCounter] = shiftRegister;
47
48             // Incremento del contador (Buffer circular 0-15)
49             addressCounter = (addressCounter + 1) % 16;
50
51             q1 = 0; q0 = 0; // Retorno a IDLE (00)
52             break;
53     }
54     delay(10); // Retardo emulando el ciclo de reloj
55 }
```

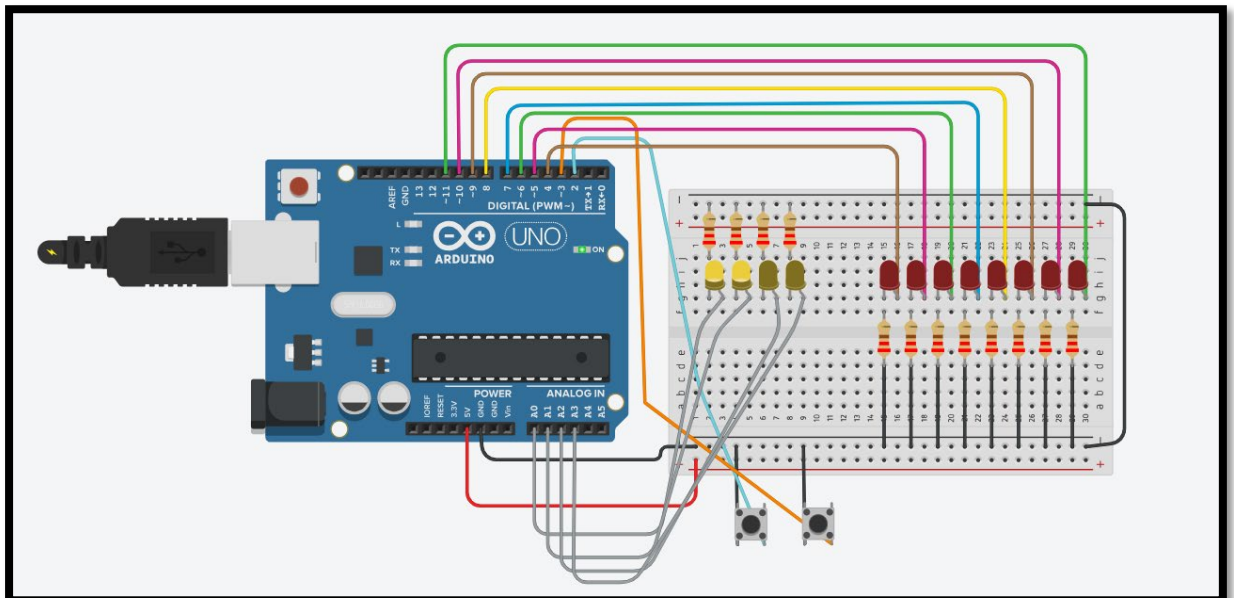
```
30
31     case 1: // START (01)
32         q1 = 1; q0 = 0; // Transición incondicional a DATA (10)
33         break;
34
35     case 2: // DATA (10) - Habilitación del Registro de Desplazamiento
36         // Captura de datos emulada (Desplazamiento a la izquierda)
37         shiftRegister = (shiftRegister << 1) | rx;
38
39         if (c8 == 1) { // Si se completan los 8 bits
40             q1 = 1; q0 = 1; // Salto a SAVE (11)
41         }
42         break;
43
44     case 3: // SAVE (11) - Write Enable activado
45         // El decodificador selecciona la posición addressCounter para guardar el byte
46         ramBuffer[addressCounter] = shiftRegister;
47
48         // Incremento del contador (Buffer circular 0-15)
49         addressCounter = (addressCounter + 1) % 16;
50
51         q1 = 0; q0 = 0; // Retorno a IDLE (00)
52         break;
53 }
54 delay(10); // Retardo emulando el ciclo de reloj
55 }
```

6. Plan de simulación

Para asegurar una cobertura funcional superior al 90%, el plan de simulación contempla la inyección de los siguientes estímulos críticos:

- **Caso de Prueba 1 (Reposo y Rechazo de Ruido):** Se inyecta $RX=1$ constante. Justificación: Verifica que la ecuación de D0 mantiene la FSM enclavada en IDLE sin habilitar el registro ni corromper la memoria.
- **Caso de Prueba 2 (Arranque y Desplazamiento):** Se fuerza un flanco de bajada $RX=0$ seguido de una ráfaga de datos 10101010 alternando el reloj. Justificación: Valida la transición $01 \rightarrow 10$ y comprueba que el Enable del registro funciona correctamente acoplado a Q1.
- **Caso de Prueba 3 (Escritura y Retorno):** Estando en DATA (10), se activa $C8=1$. Justificación: Caso límite que evalúa si la memoria RAM captura el byte en la dirección correcta del decodificador y si el contador incrementa su valor al retornar asíncronamente a IDLE.

7. Simulación



Al transitar la FSM al estado 11 (SAVE), la señal combinacional del Decodificador se alinea con el pulso del reloj, generando la señal Write Enable. Esto produce el volcado exacto del bus de datos del registro de desplazamiento (0xAA) hacia la matriz de la RAM antes de que el contador modifique la dirección en el siguiente flanco de bajada.

8. Análisis teórico de las métricas (fórmulas, valores esperados)

8.1. Frecuencia Máxima de Operación (F_{max}):

Determina la velocidad de reloj límite a la que el sistema puede operar garantizando la estabilidad de las señales digitales a través de la lógica combinacional de la FSM y los registros. Se calcula mediante la suma de los tiempos de propagación de los bloques críticos:

$$F_{max} = \frac{1}{T_{pd(FSM)} + T_{su(Registro)} + T_{clk_q}}$$

Donde:

- $T_{pd(FSM)}$: Retardo de propagación de la lógica combinacional de estado siguiente.
- $T_{su(Registro)}$: Tiempo de *setup* del registro de desplazamiento.
- T_{clk_q} : Tiempo de propagación desde el flanco de reloj hasta la salida del biestable.

Valor esperado: > 50 MHz en implementación sobre FPGA, lo cual supera ampliamente las velocidades requeridas por los estándares UART industriales (desde 9600 hasta 115200 bps), asegurando una operación holgada y confiable.

8.2. Latencia de Almacenamiento (T_{write}):

Es el tiempo requerido por el sistema para completar el ciclo de escritura en la memoria una vez recibida la trama completa. Este tiempo es una medida directa de la velocidad de respuesta de la unidad de control ante el evento de fin de byte (C8=1).

$$T_{write} = 2 \times T_{clk}$$

Valor esperado: $T_{write} \leq 2$ ciclos de reloj.

Análisis: El primer ciclo corresponde a la transición de estado desde DATA (10) hacia SAVE (11) al detectar la señal C8, y el segundo ciclo corresponde a la activación de la señal Write Enable y la estabilización del dato en la celda de memoria seleccionada por el decodificador. Este tiempo es constante y determinístico, lo que garantiza una gestión de datos predecible en el buffer circular.